

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

*I **C. PRIYANKA(192511074)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: C. PRIYANKA

Design Task

1. **Hybrid AI Recommendation Engine Design**
Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**
Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**
Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**
Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**
Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

ANSWERS

1) Hybrid AI Recommendation Engine Design

1. Problem Understanding & Requirement Analysis

Problem

Large e-commerce and entertainment platforms have millions of users and items. Providing personalized recommendations requires processing huge amounts of user-item interaction data.

Objectives

- Build a hybrid recommendation system.
- Combine **Content-Based Filtering** and **Collaborative Filtering**.
- Process large datasets using PySpark RDDs.
- Adapt recommendations based on user behavior.
- Provide real-time personalized recommendations.

Constraints

- Very large user-item datasets.
- Changing user preferences.
- Cold-start problem for new users/items.
- Low recommendation response time.

2. System Architecture

Users

|

v

User-Item Interaction Data

|

v

Data Collection

|

v

PySpark RDD Processing

|

+-----+

|

v

|

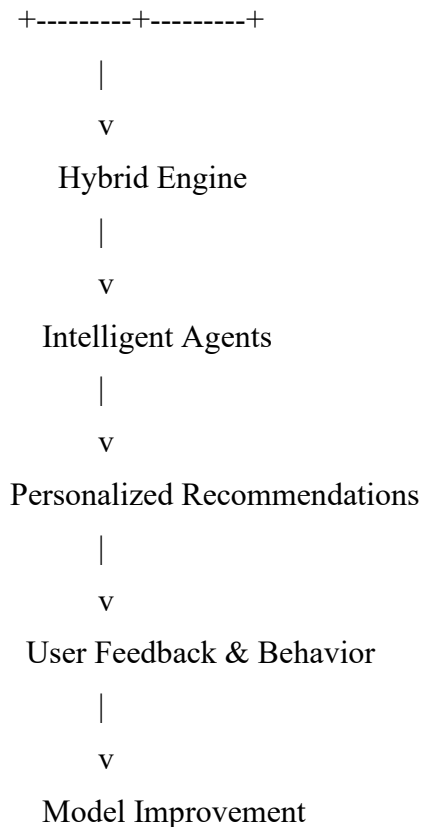
v

Content-Based Collaborative

Filtering Filtering

|

|



Main Components

1. **Data Collection** – collects ratings, clicks, purchases and searches.
2. **RDD Processing** – distributes user-item data across worker nodes.
3. **Content-Based Filter** – recommends items similar to previously liked items.
4. **Collaborative Filter** – finds users/items with similar behavior.
5. **Hybrid Engine** – combines both recommendation results.
6. **Intelligent Agents** – monitor user behavior and modify recommendations.
7. **Feedback Module** – uses clicks, ratings and purchases to improve recommendations.

3. PySpark & RDD Operations

```
data = sc.textFile("ratings.csv")
```

```
records = data.map(lambda x: x.split(","))
```

```
valid = records.filter(lambda x: len(x) >= 4)
```

```
user_items = valid.map(  
    lambda x: (x[0], x[1], float(x[2]))  
)
```

For aggregation:

```
ratings = user_items.map(
    lambda x: (x[0], x[2])
).reduceByKey(lambda a, b: a + b)
```

Important RDD operations:

Operation	Purpose
map()	Transform user-item records
filter()	Remove invalid records
flatMap()	Generate multiple recommendation features
groupByKey()	Group items by users
reduceByKey()	Aggregate ratings
sortBy()	Rank recommendations

4. Intelligent Agent Integration

User Behavior Agent:

Tracks clicks, searches, purchases and ratings.

Recommendation Agent:

Generates recommendations using both filtering methods.

Feedback Agent:

Analyzes whether the user accepts or rejects recommendations.

Adaptation Agent:

Updates recommendations according to new user behavior.

User Activity



Behavior Agent



Recommendation Agent



Hybrid Recommendation



User Feedback



Feedback Agent



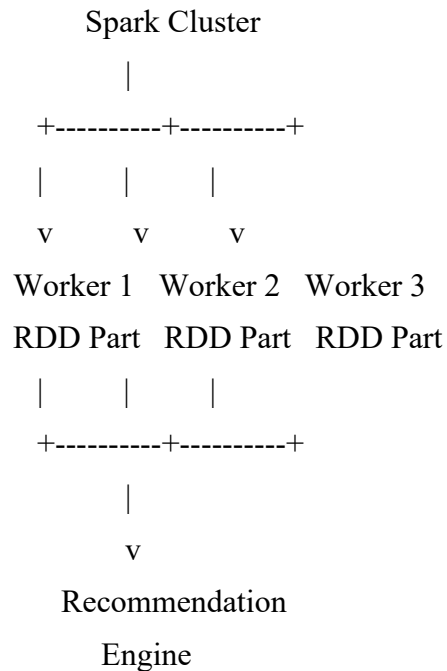
Adaptation Agent



Improved Recommendation

5. Scalability & Distributed Computing

PySpark divides the large user-item dataset into RDD partitions.



This provides:

- Parallel processing.
- Faster recommendation generation.
- Fault tolerance.
- Easy addition of worker nodes.
- Processing of millions of interactions.

6. Data Flow

User Data



Data Collection



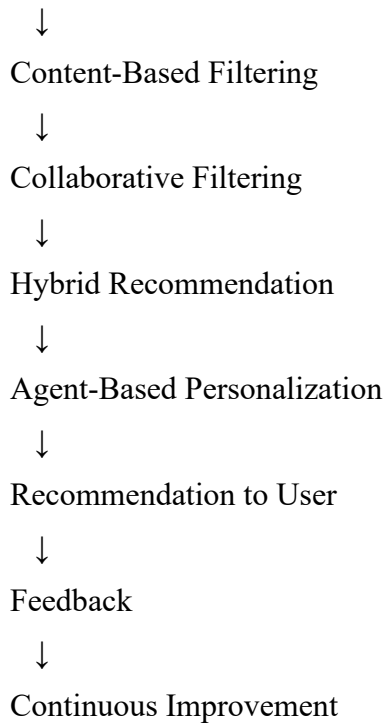
RDD Creation



Data Cleaning



Feature Extraction



7. Innovation

The system combines **content-based filtering + collaborative filtering + PySpark distributed processing + intelligent agents**. This improves personalization and allows recommendations to adapt dynamically.

Conclusion

The Hybrid AI Recommendation Engine can process millions of user-item interactions using PySpark RDDs. Intelligent agents continuously analyze user behavior and feedback, allowing the system to provide adaptive and personalized recommendations.

2) Smart Disaster Detection System

1. Problem Understanding & Requirement Analysis

Problem

Natural disasters such as floods and earthquakes can occur suddenly. Data is generated from satellites, sensors and social media, but processing these sources quickly is difficult.

Objectives

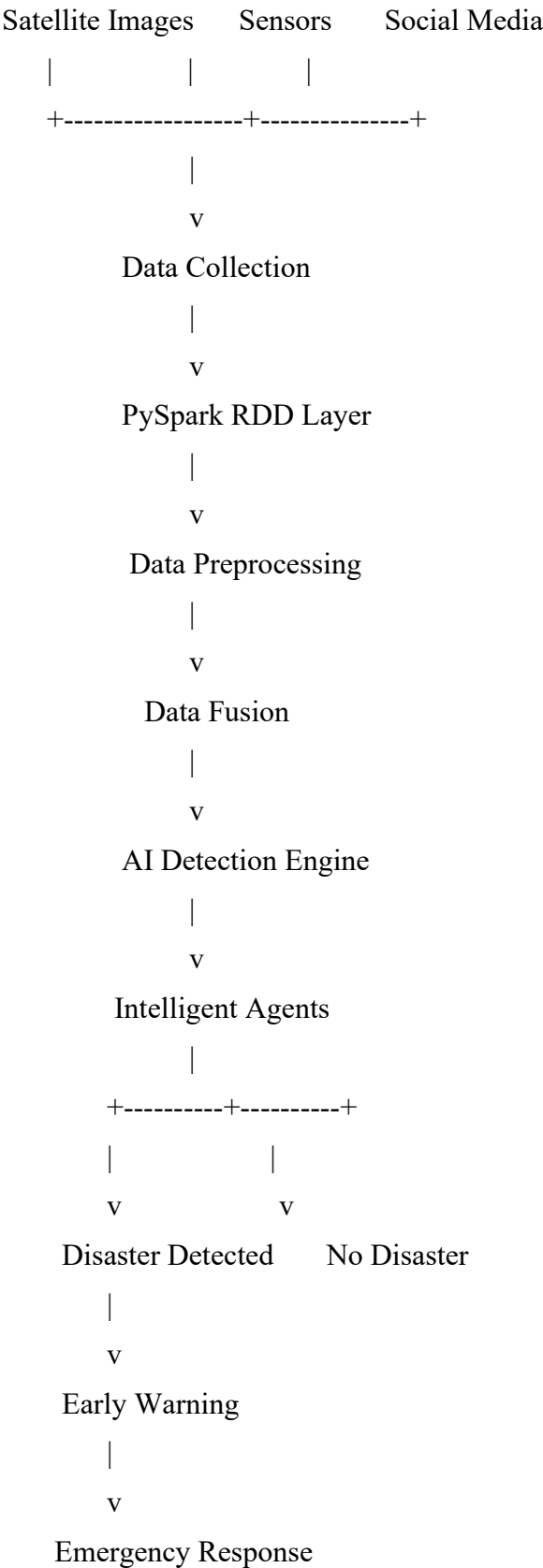
- Detect disasters at an early stage.
- Process multi-source data using PySpark RDDs.
- Combine satellite, sensor and social media information.
- Generate early warnings.
- Support emergency response decisions.

Constraints

- Large-volume data.
- Real-time processing requirements.

- Noisy or incomplete data.
- Need for high detection accuracy.

2. System Architecture



3. Data Fusion

Different data sources are combined to improve detection accuracy.

Example

Satellite Data

+

Sensor Data

+

Social Media Data

↓

Multi-Source Data Fusion

↓

Disaster Probability

For example, if water-level sensors report increasing water levels, satellite images show flooded areas and social media contains flood-related posts, the system can increase the probability of a flood.

4. PySpark & RDD Operations

```
satellite = sc.textFile("satellite_data")
```

```
sensors = sc.textFile("sensor_data")
```

```
social = sc.textFile("social_data")
```

```
sensor_data = sensors.map(lambda x: x.split(","))
```

```
high_water = sensor_data.filter(  
    lambda x: float(x[2]) > 80  
)
```

RDD operations such as map(), filter(), union(), groupByKey() and reduceByKey() can combine and analyze large-scale disaster data.

5. Intelligent Agent Integration

Monitoring Agent:

Monitors sensors and incoming data.

Image Analysis Agent:

Analyzes satellite information for flood or earthquake indicators.

Social Media Agent:

Identifies disaster-related information from social media.

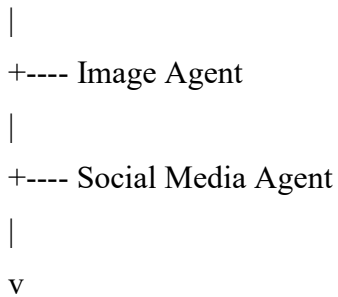
Warning Agent:

Generates warnings when disaster probability crosses a threshold.

Response Agent:

Suggests evacuation routes, rescue priorities and emergency resources.

Monitoring Agent



Data Fusion



Warning Agent



Response Agent

6. Scalability

PySpark distributes satellite, sensor and social media data across multiple workers.

Benefits:

- Parallel processing.
- Fast disaster detection.
- Fault tolerance.
- Large-scale data handling.
- Easy expansion by adding nodes.

7. Data Flow

Multi-Source Data



Data Collection



RDD Creation



Preprocessing



Data Fusion



AI Disaster Detection



Agent Coordination



Disaster Probability



Early Warning



Emergency Response

8. Innovation

The system combines **satellite imagery, IoT sensors, social media, distributed AI and intelligent agents**. Using multiple sources reduces the possibility of relying on a single unreliable data source.

Conclusion

The Smart Disaster Detection System provides a scalable distributed solution for detecting disasters early. PySpark handles large data volumes while intelligent agents coordinate detection, warning and response activities.

3) AI-Based Fraud Detection Framework

1. Problem Understanding & Requirement Analysis

Problem

Banks and financial institutions process millions of transactions. Fraudsters continuously develop new techniques, making traditional rule-based detection insufficient.

Objectives

- Detect fraudulent transactions.
- Process large transaction datasets using PySpark.
- Identify abnormal patterns.
- Generate real-time alerts.
- Improve detection using feedback.

Constraints

- High transaction volume.
- Real-time detection.
- Need for high accuracy.
- Reduction of false positives.
- Security and privacy requirements.

2. System Architecture

Financial Transactions

|

v

Data Collection

|

v

PySpark RDD

|

v

Data Cleaning & Feature Extraction

|

v

Anomaly Detection

|

v

Intelligent Agent Layer

|

+--+--+

| |

v v

Normal Fraud

| |

| v

| Alert

| |

+-----+

|

v

Feedback

|

v

Model Improvement

3. PySpark & RDD Operations

```
transactions = sc.textFile("transactions.csv")
```

```
data = transactions.map(lambda x: x.split(","))
```

```
valid = data.filter(lambda x: float(x[2]) > 0)
```

```
features = valid.map(  
    lambda x: (x[0], float(x[2]), x[3], x[4])  
)
```

Aggregation:

```
customer_total = features.map(  
    lambda x: (x[0], x[1])  
)reduceByKey(lambda a, b: a + b)
```

RDD operations:

- map()
- filter()
- flatMap()
- reduceByKey()
- groupByKey()
- sortBy()

4. Anomaly Detection

The system checks:

- Unusually high transaction amounts.
- Unusual transaction locations.
- Rapid transaction frequency.
- Unusual device usage.
- Abnormal purchasing behavior.

Normal Behavior → Low Fraud Score → Approve

Abnormal Behavior → High Fraud Score → Alert

5. Intelligent Agents

Monitoring Agent:

Monitors incoming transactions.

Fraud Analysis Agent:

Calculates anomaly/fraud scores.

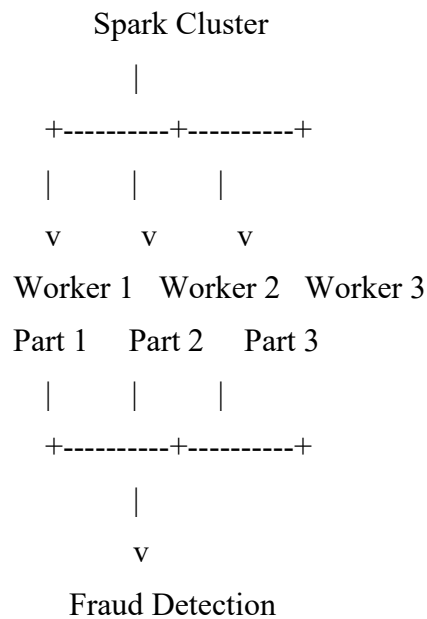
Alert Agent:

Generates alerts for suspicious transactions.

Learning Agent:

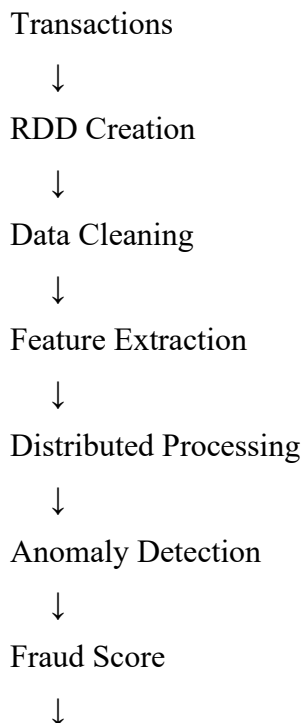
Uses confirmed fraud information to improve future detection.

6. Scalability



RDD partitioning allows transactions to be processed in parallel.

7. Data Flow



Agent Analysis



Alert / Approval



Feedback



Continuous Learning

8. Innovation

The framework combines **distributed big-data processing, anomaly detection and intelligent agents**. Agents can independently monitor, analyze and respond to suspicious activities.

Conclusion

The AI-Based Fraud Detection Framework can process millions of financial transactions efficiently. PySpark provides scalability while intelligent agents enable fast detection, alert generation and continuous improvement.

4) Autonomous Supply Chain Optimization System

1. Problem Understanding & Requirement Analysis

Problem

Supply chains involve inventory, suppliers, transportation, warehouses and customer demand. Poor coordination can cause overstocking, shortages, delivery delays and increased costs.

Objectives

- Optimize inventory levels.
- Predict customer demand.
- Improve transportation decisions.
- Minimize operational costs.
- Coordinate multiple intelligent agents.

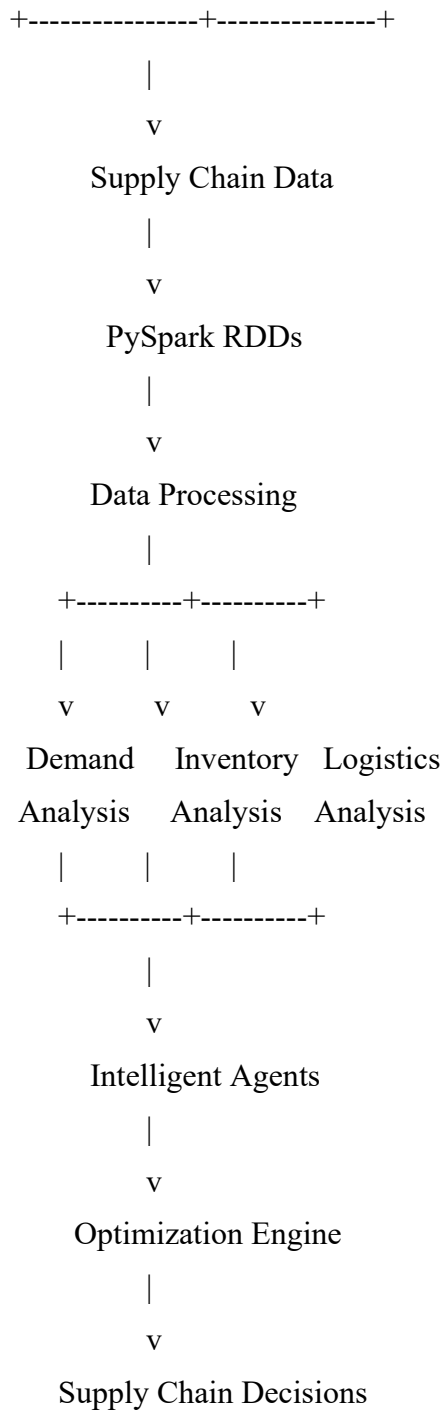
Constraints

- Changing customer demand.
- Transportation delays.
- Limited warehouse capacity.
- Supplier availability.
- Large-scale supply chain data.

2. System Architecture

Suppliers Warehouses Customers





3. PySpark & RDD Operations

```
sales = sc.textFile("sales.csv")
```

```
data = sales.map(lambda x: x.split(","))
```

```
valid_sales = data.filter(
    lambda x: float(x[2]) > 0
)
```


Calculate product demand:

```
demand = valid_sales.map(  
    lambda x: (x[1], float(x[2])))  
)reduceByKey(lambda a, b: a + b)
```

RDD operations such as map(), filter(), reduceByKey(), groupByKey() and sortBy() can process large-scale supply chain data.

4. Intelligent Agent Integration

Inventory Agent:

Monitors stock levels and recommends replenishment.

Demand Forecasting Agent:

Analyzes historical sales and predicts future demand.

Logistics Agent:

Selects efficient transportation routes.

Supplier Agent:

Monitors supplier performance and availability.

Cost Optimization Agent:

Compares alternatives and selects cost-efficient decisions.

Demand Agent

|

Inventory Agent

|

Logistics Agent

|

Supplier Agent

|

v

Optimization Agent

|

v

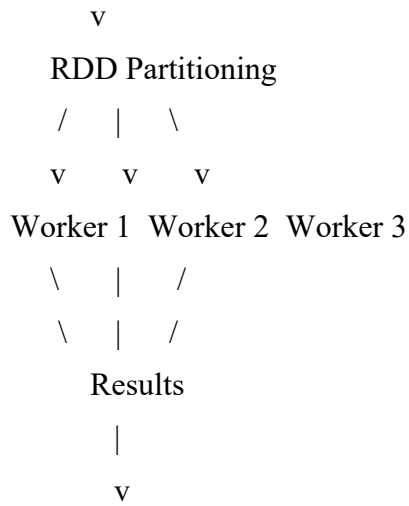
Final Supply Chain Decision

5. Scalability

Large sales and logistics datasets are divided into RDD partitions.

Large Supply Chain Dataset

|



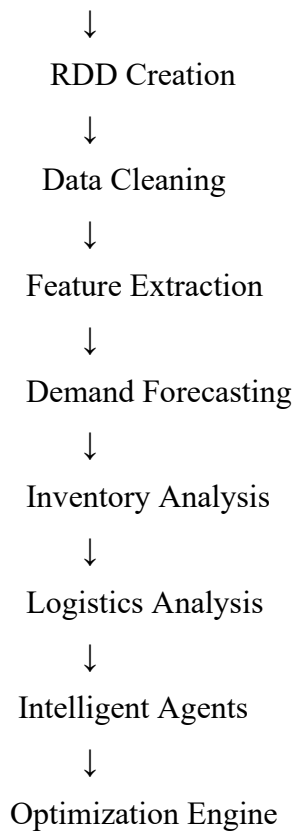
Optimization

Advantages:

- Parallel processing.
- Faster demand analysis.
- Fault tolerance.
- Large-scale data handling.
- Easy cluster expansion.

6. Data Flow

Supplier/Sales/Inventory Data





Final Decision



Cost Reduction & Efficiency

7. Innovation

The system uses **multiple autonomous agents** instead of depending on a single decision-making module. Agents coordinate inventory, demand, logistics and suppliers to achieve overall cost minimization.

Conclusion

The Autonomous Supply Chain Optimization System uses PySpark for scalable data processing and intelligent agents for decentralized decision-making. It can reduce inventory costs, improve delivery efficiency and respond quickly to demand changes.

5) Personalized Learning Analytics Platform

1. Problem Understanding & Requirement Analysis

Problem

Educational platforms generate large amounts of student data such as attendance, quiz scores, assignment performance, learning time and interaction history. A single learning approach may not suit every student.

Objectives

- Analyze student learning behavior.
- Predict student performance.
- Identify weak learning areas.
- Provide personalized learning content.
- Give real-time feedback.
- Support thousands of learners.

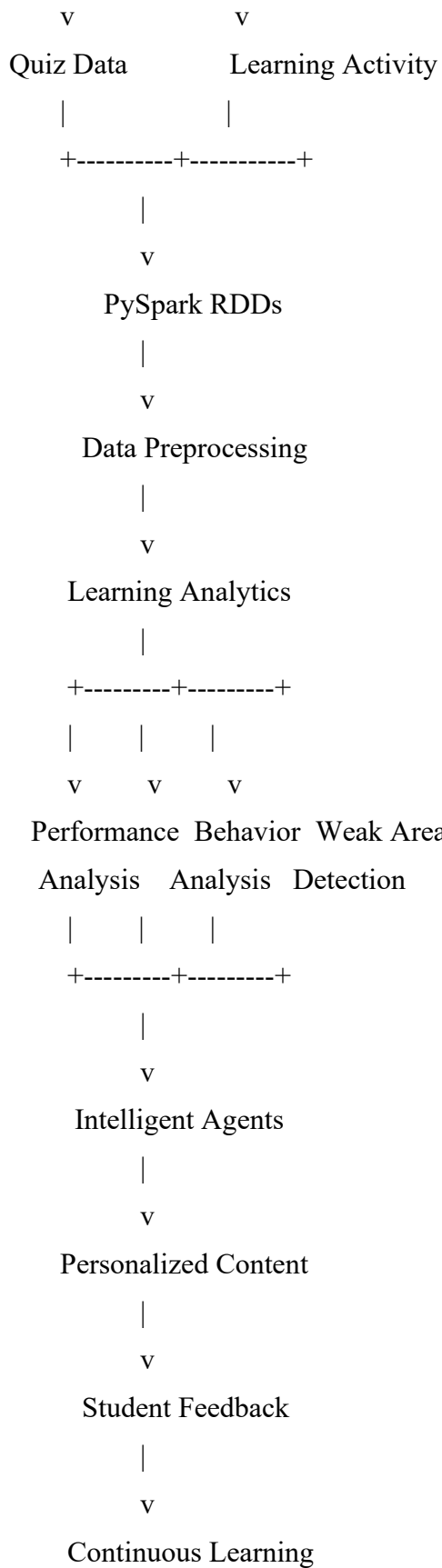
Constraints

- Large student datasets.
- Different learning speeds.
- Changing student behavior.
- Need for personalized recommendations.
- Privacy of student information.

2. System Architecture

Student Activities





3. PySpark & RDD Operations

```
students = sc.textFile("student_data.csv")
```

```
data = students.map(lambda x: x.split(","))
```

```
valid_data = data.filter(  
    lambda x: len(x) >= 5  
)
```

Calculate average performance:

```
scores = valid_data.map(  
    lambda x: (x[0], float(x[2])))  
)
```

```
average = scores.groupByKey().mapValues(  
    lambda values: sum(values) / len(values)  
)
```

Useful RDD operations:

RDD Operation Purpose

map()	Transform student records
filter()	Remove invalid data
groupByKey()	Group activities by student
reduceByKey()	Calculate aggregate performance
sortBy()	Rank student performance
mapValues()	Calculate student-specific metrics

4. Intelligent Agent Integration

Student Monitoring Agent:

Tracks student activity, attendance and learning time.

Performance Agent:

Analyzes quiz and assignment performance.

Content Recommendation Agent:

Suggests suitable learning materials.

Feedback Agent:

Provides immediate feedback.

Adaptive Learning Agent:

Changes difficulty level and content according to student progress.

Student Activity



Monitoring Agent



Performance Agent



Weak Area Detection



Content Recommendation Agent



Personalized Content



Student Feedback



Adaptive Learning Agent



Updated Learning Path

5. Scalability

Thousands of student records can be divided into RDD partitions and processed simultaneously.

Student Dataset



RDD Dataset



Worker 1 Worker 2 Worker 3



Learning Analytics



Personalized Results

Benefits:

- Supports thousands of learners.
- Parallel processing.
- Faster analytics.
- Fault tolerance.
- Easy addition of computing nodes.

6. Data Flow

Student Data



Data Collection



RDD Creation



Data Cleaning



Feature Extraction



Performance Analysis



Behavior Analysis



Weak Area Identification



Intelligent Agent Processing



Personalized Content



Real-Time Feedback



Student Response



Adaptive Learning

7. Innovation

The platform combines **big-data analytics, PySpark RDDs, intelligent agents and adaptive learning**. Instead of providing identical content to every student, the system dynamically adjusts learning materials based on individual performance and behavior.

Conclusion

The Personalized Learning Analytics Platform provides scalable and adaptive education for thousands of learners. PySpark handles large volumes of student data, while intelligent agents analyze learning behavior, recommend content and provide continuous feedback.